

CS 224

Lab 03

Section: 04

Nabeeha Khan

22301304

12th November 2025

Part (B): Assembly Language Equivalent

Address	Instruction (in Hex)	Mips Instruction
0x00000000	0x20020005	addi \$v0, \$zero, 0x0005
0x00000004	0x2003000C	addi \$v1, \$zero, 0x000C
0x00000008	0x2067FFF7	addi \$a3, \$v1, 0xFFFF7
0x0000000C	0x00E22025	or \$a0, \$a3, \$v0
0x00000010	0x00642824	and \$a1, \$v1, \$a0
0x00000014	0x00A42820	add \$a1, \$a1, \$a0
0x00000018	0x10A7000A	beq \$a1, \$a3, 0x00000044
0x0000001C	0x0064202A	slt \$a0, \$v1, \$a0
0x00000020	0x10800001	beq \$a0, \$zero, 0x00000028
0x00000024	0x20050000	addi \$a1, \$zero, 0x0000
0x00000028	0x00E2202A	slt \$a0, \$a3, \$v0
0x0000002C	0x00853820	add \$a3, \$a0, \$a1
0x00000030	0x00E23822	sub \$a3, \$a3, \$v0
0x00000034	0xAC670044	sw \$a3, 0x0044(\$v1)
0x00000038	0x8C020050	lw \$v0, 0x0050(\$zero)
0x0000003C	0x08000011	j 0x00000044
0x00000040	0x20020001	addi \$v0, \$zero, 0x0001
0x00000044	0xAC020054	sw \$v0, 0x0054(\$zero)
0x00000048	0x08000012	j 0x00000048

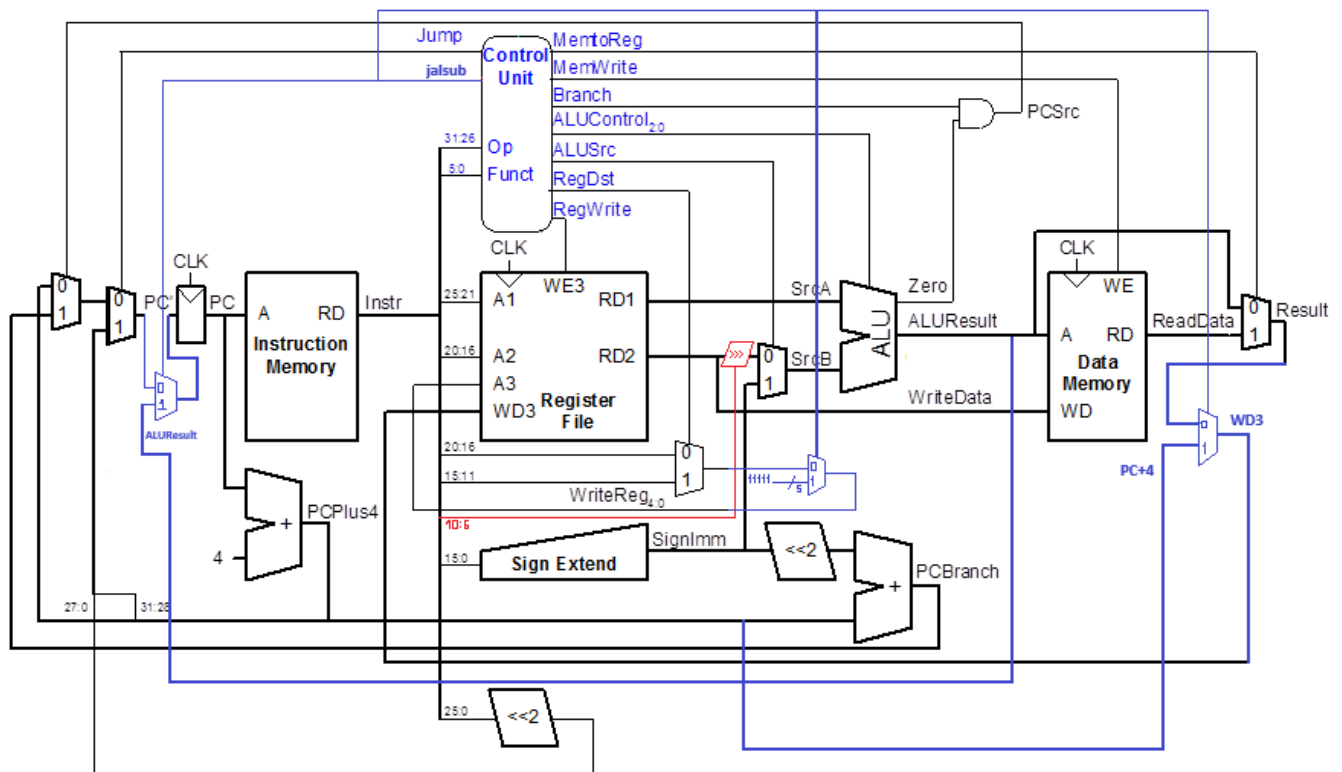
Part (C): RTL Expressions

sra \$rd, \$rt, shamt: $R[rd] = R[rt] \ggg shamt$

jalsub \$rs, \$rt: $R[31] = PC + 4; PC = R[rs] - R[rt]$

Part (D): Datapath

Note: Changes marked with red are for sra instruction and changes marked with blue are for jalsub instruction



Part (E): Control Table

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemToReg	ALUOp	Jump	Jalsub
R-type	000000	1	1	0	0	0	0	10	0	0
lw	100011	1	0	1	0	0	1	00	0	0
sw	101011	0	X	1	0	1	X	00	0	0
beq	000100	0	X	0	1	0	X	01	0	0
addi	001000	1	0	1	0	0	0	00	0	0
j	000010	0	X	X	X	0	X	XX	1	0
sra	000000	1	1	0	0	0	0	10	0	0
jalsub	000000	1	X	0	X	0	X	10	X	1

Table 1: Main decoder

ALUOp	Funct	ALUControl
00	X	010 (add)
01	X	110 (subtract)
1X	100000 (add)	010 (add)
1X	100010 (sub)	110 (subtract)
1X	100100 (and)	000 (and)
1X	100101 (or)	001 (or)
1X	101010 (slt)	111 (set let than)
1X	000011 (sra)	010 (add)
1X	000101 (jalsub)	110 (subtract)

Part (F): Test Program

addi \$a0, \$zero, 8

addi \$a1, \$zero, 4

lw \$ra, 8(\$v0)

add \$s0, \$a0, \$a1

or \$s1, \$a0, \$a1

and \$s2, \$a0, \$a1

sra \$s3, \$s0, 2

sub \$a0, \$a0, \$a1

slt \$at, \$a0, \$a1

beq \$at, \$zero, end

```

beq $a0, $a1, end
jalsub $a0, $a1
end: sw $ra, 8($v0)
j end

```

Part (G): System Verilog Modules

Changes Made:

- In imem module, where we add new instr to test jalsub and sra.
- In mips module, where controller c returns an additional output jalsub and datapath dp takes jalsub as input.
- In controller module, where it returns an additional output jalsub, and maindec md will take funct as input and return jalsub as output.
- In maindec module, where we will compute an additional jalsub.
- In aludec module, where we add two additional conditions for jalsub and sra.
- In datapath module, jalsub is taken as input. Three additional muxs all with signal jalsub: one decides between output from jump mux and ALU output, one decides between register 31 (\$ra) and output of mux deciding between \$rd and \$rt, one decides between result and pc+4. Also, before the mux deciding between register and imm value, sra is created that returns shifted value of register.
- Creating an additional module sra, for arithmetic right shifting.

Code:

```

module imem ( input logic [7:0] addr, output logic [31:0] instr);

  always_comb

    case (addr)    // word-aligned fetch
    // address instruction
    // -----
    8'h00: instr = 32'h 20040008;
    8'h04: instr = 32'h20050004;
    8'h08: instr = 32'h 8C5F0008;

```

```
8'h0c: instr = 32'h00858020;
```

```
8'h10: instr = 32'h00858825;
```

```
8'h14: instr = 32'h00859024;
```

```
8'h18: instr = 32'h00109883;
```

```
8'h1c: instr = 32'h00852022;
```

```
8'h20: instr = 32'h0085082A;
```

```
8'h24: instr = 32'h10200002;
```

```
8'h28: instr = 32'h10850001;
```

```
8'h2c: instr = 32'h00850005;
```

```
8'h30: instr = 32'hAC5F0008;
```

```
8'h34: instr = 32'h0200000D;
```

```
    default: instr = {32{1'bx}}; // unknown address
```

```
endcase
```

```
endmodule
```

```
// single-cycle MIPS processor, with controller and datapath
```

```
module mips (input logic clk, reset,
```

```
    output logic[31:0] pc,
```

```
    input logic[31:0] instr,
```

```
    output logic memwrite,
```

```
    output logic[31:0] aluout, writedata,
```

```
    input logic[31:0] readdata);
```

```
logic memtoreg, pcsrc, zero, alusrc, regdst, regwrite, jump;
```

```
logic [2:0] alucontrol;
```

```
controller c (instr[31:26], instr[5:0], zero, memtoreg, memwrite, pcsrc,
```

```

        alusrc, regdst, regwrite, jump, jalsub, alucontrol);
datapath dp (clk, reset, memtoreg, pcsrc, alusrc, regdst, regwrite, jump, jalsub,
        alucontrol, zero, pc, instr, aluout, writedata, readdata);
endmodule

```

//Controller

```

module controller(input logic[5:0] op, funct,
        input logic    zero,
        output logic    memtoreg, memwrite,
        output logic    pcsrc, alusrc,
        output logic    regdst, regwrite,
        output logic    jump, jalsub
        output logic[2:0] alucontrol);
    logic [1:0] aluop;
    logic branch;
    maindec md (op, funct, memtoreg, memwrite, branch, alusrc, regdst, regwrite,
    jump, jalsub, aluop); // Decoder for the multiplexer signals per instr
    aludec ad (funct, aluop, alucontrol); // Decoder for the ALU signals per instr
    assign pcsrc = branch & zero; // Signal for the PC's branch multiplexer
endmodule

```

//Main Decoder

```

module maindec (input logic[5:0] op, funct
        output logic memtoreg, memwrite, branch,
        output logic alusrc, regdst, regwrite, jump, jalsub,

```

```

        output logic[1:0] aluop );

    logic [9:0] controls;

    //Assigning to all the output signals with one variable to make assignments easy
    assign {regwrite, regdst, alusrc, branch, memwrite,
            memtoreg, aluop, jump, jalsub} = controls;

    //This block shows the main actions of the controller according to the opcode
    always_comb

    //decide on regwrite, regdst, alusrc, branch, memwrite, memtoreg, aluop, jump,
    //jalsub in this order.

    case(op)
        6'b000000: if (funct == 6'b000101)
            controls <= 10'b1100001000; // R-type + SRA
        else
            controls <= 10'b1x0x0x10x1; // JALSUB
        6'b100011: controls <= 10'b1010010000; // LW
        6'b101011: controls <= 10'b0010100000; // SW
        6'b000100: controls <= 10'b0001000100; // BEQ
        6'b001000: controls <= 10'b1010000000; // ADDI
        6'b000010: controls <= 10'b0000000010; // J
        default: controls <= 10'bxxxxxxxxxx; // illegal op
    endcase
endmodule

//ALU decoder
module aludec (input  logic[5:0] funct,
               input  logic[1:0] aluop,

```



```

        output logic[2:0] alucontrol);
always_comb
// aludec uses funct and aluop to decide on the output bits
// aluop encoding can be seen in the table 7.1
// aludec truth table can be seen in the table 7.2
case(aluop)
    2'b00: alucontrol = 3'b010; // add (for lw/sw/addi)
    2'b01: alucontrol = 3'b110; // sub (for beq)
    default: case(funct) // R-TYPE instructions
        6'b100000: alucontrol = 3'b010; // ADD
        6'b100010: alucontrol = 3'b110; // SUB
        6'b100100: alucontrol = 3'b000; // AND
        6'b100101: alucontrol = 3'b001; // OR
        6'b101010: alucontrol = 3'b111; // SLT
        6'b000011: alucontrol = 3'b010; //SRA
        6'b000101: alucontrol = 3'b110; //JALSUB
        default: alucontrol = 3'bxxx; // ???
    endcase
endcase
endmodule

module datapath (input logic clk, reset, memtoreg, pcsrc, alusrc, regdst,
                 input logic regwrite, jump, jalsub
                 input logic[2:0] alucontrol,
                 output logic zero,

```

```

        output logic[31:0] pc,
        input  logic[31:0] instr,
        output logic[31:0] aluout, writedata,
        input  logic[31:0] readdata);

logic [4:0] writereg;
logic [31:0] pcnext, pcnextbr, pcnextj, pcplus4, pcbranch;
logic [31:0] signimm, signimmsh, srca, srcb, result;

// next PC logic
flopr #(32) pcreg(clk, reset, pcnext, pc);
adder    pcadd1(pc, 32'b100, pcplus4);
sl2      immsh(signimm, signimmsh);
adder    pcadd2(pcplus4, signimmsh, pcbranch);
mux2 #(32) pcbrmux(pcplus4, pcbranch, pcsrc,
                  pcnextbr);
mux2 #(32) pcjmux(pcnextbr, {pcplus4[31:28],
                             instr[25:0], 2'b00}, jump, pcnextj);

// register file logic
regfile  rf (clk, regwrite, instr[25:21], instr[20:16], writereg,
            result, srca, writedata);

mux2 #(5) wrmux (instr[20:16], instr[15:11], regdst, writeregtemp);
mux2 #(5) wrmux1 (writeregtemp, 5'b11111, jalsub, writereg);
mux2 #(32) resmux (aluout, readdata, memtoreg, resulttemp);
mux2 #(32) resmux1 (resulttemp, pcplus4, jalsub, result);

signext  se (instr[15:0], signimm);

// ALU logic

```

```

sra sra(writedata, instr[10:6], writedatashifted);
    mux2 #(32) srcbmux (writedatashifted, signimm, alusrc, srcb);
    alu      alu (srca, srcb, alucontrol, aluout, zero);
//Mux to pick between pcnextj or ALUResult
    mux2 #(32) pcmux(pcnextj, aluout, jalsub,
                    pcnext);

```

```

endmodule

```

```

// Arithmetic Right Shifter
module sra(input logic [31:0] a,
           input logic [4:0] shamt,
           output logic [31:0] y);
    always_comb begin
        y = a >>> shamt;
    end
endmodule

```